

Idempotency: Make retries safe.

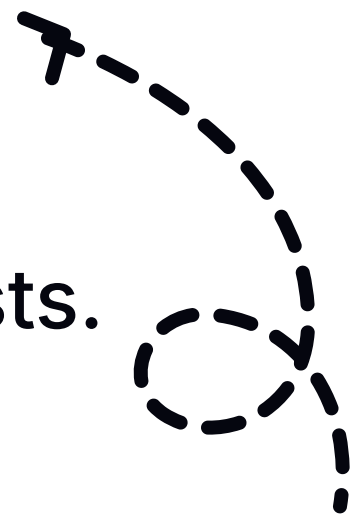
Networks fail. Clients retry.
Your system should be ready
for **both.**



Peter Kekpe
@peter-kekpe

One customer. **Three** charges.

The client never sent three requests.
It sent one, then retried twice.



Peter Kekpe
@peter-kekpe

SWIPE



03. WHAT ACTUALLY HAPPENED 3/9

The response died, not the write

`POST /payments → 201 • response lost`

Your server committed the charge. The connection dropped before the response came back. The client saw a timeout and did the only sensible thing it could. It retried.

01 Attempt 1: charge committed, response lost

02 Attempt 2: retry, charged again

03 Attempt 3: retry, charged again



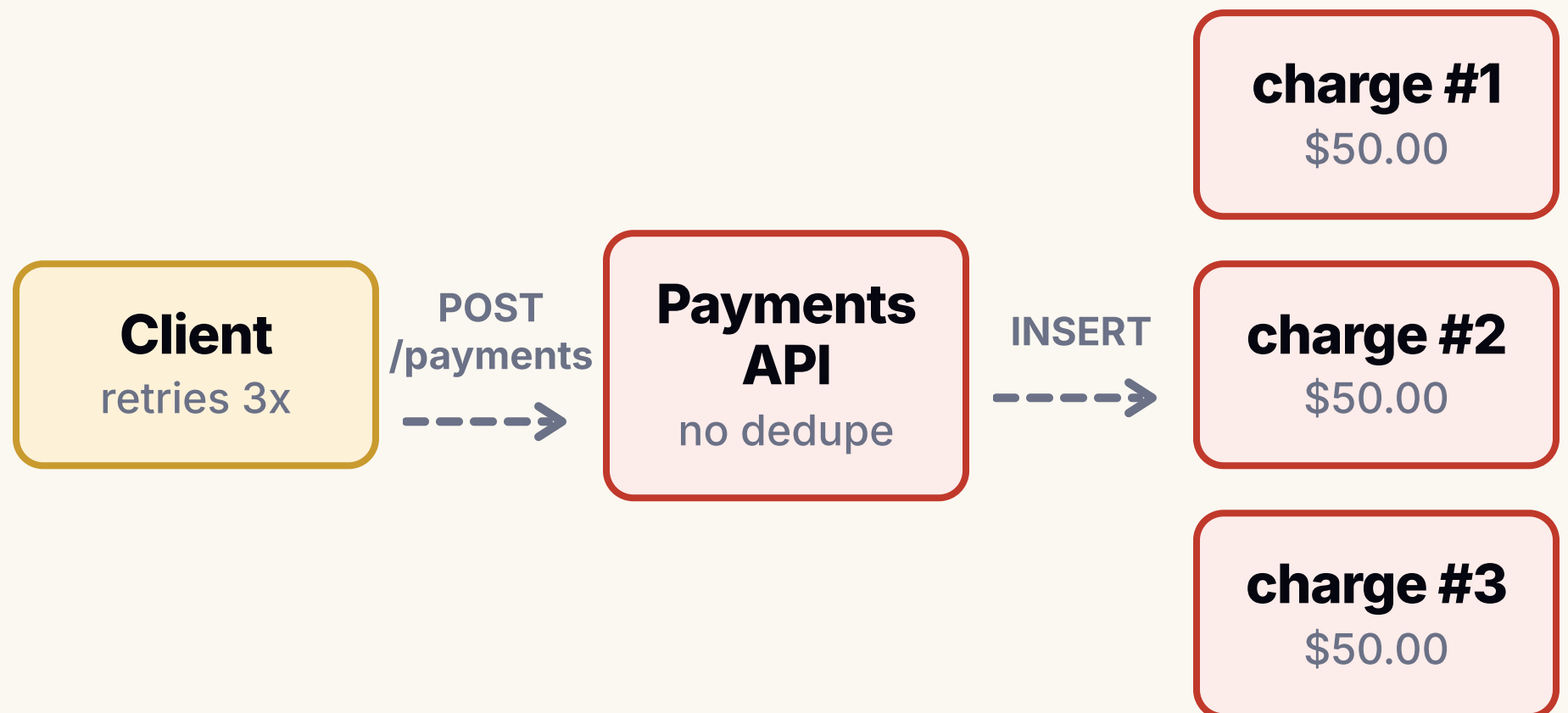
Peter Kekpe
@peter-kekpe

SWIPE

The retry storm

4/9

A timeout is not a failure.



A timeout tells you the response is missing. It never tells you the write didn't happen.



Peter Kekpe
@peter-kekpe

SWIPE



05. THE FIX

5/9

Give every attempt a name

One key per **intent**, not per attempt. The same key rides every retry.

WITHOUT A KEY

POST /payments



Every retry looks like a brand new order.

WITH A KEY

Idempotency-Key: 7c9e-4f21



Every retry resolves to the same order.



Peter Kekpe
@peter-kekpe

SWIPE



06. THE BUG IN EVERY TUTORIAL

6/9

The loser has nothing to replay

20 concurrent retries → 19 saw 'in_progress'

The winner hasn't finished yet, so no response is stored. Returning **200** here tells the client the payment succeeded before it actually has.

-
- 01 Replay **only** when the stored status is `done`
 - 02 **409 Conflict** while the winner is still in flight
 - 03 This is what **Stripe** returns in the same case



Peter Kekpe
@peter-kekpe

SWIPE

Claim the key first

payments.controller.js

JS

```
const claim = await db.query(  
  `INSERT INTO idempotency_keys (key, status)  
  VALUES ($1, 'in_progress')  
  ON CONFLICT (key) DO NOTHING  
  RETURNING key`, [key]);  
  
if (!claim.rowCount) {  
  const prior = await load(key);  
  // Winner still running. Nothing to replay yet.  
  if (prior.status === 'in_progress')  
    return res.status(409);  
  return res.status(prior.code).json(prior.body);  
}
```

→ The unique index arbitrates, not your code. Claim it in its own transaction, never the one doing the charge.



Peter Kekpe
@peter-kekpe

SWIPE

Two retries, one row

schema.sql

SQL

```
CREATE TABLE idempotency_keys (  
  key          TEXT PRIMARY KEY,  
  status       TEXT NOT NULL,  
  response     JSONB,  
  created_at   TIMESTAMPTZ DEFAULT now()  
);
```

```
-- Two retries in the same millisecond both hit  
-- this index. Exactly one INSERT survives.
```

→ No lock. No queue. No distributed coordinator. One **PRIMARY KEY** does the arbitration for you.



Peter Kekpe
@peter-kekpe

SWIPE

Takeaway

**A retry is not
a new request.
It is the **same intent**,
asking again.**

Design every write endpoint so the second attempt is boring.

Backend Engineering & System Architecture, daily.
Follow @peter-kekpe for the next one.



Peter Kekpe
@peter-kekpe